Community Summaries (커뮤니티 요약)

개요

Community Summaries는 **Microsoft GraphRAG**에서 도입한 개념으로, Knowledge Graph의 **노드 클러스터(커뮤니티)**를 LLM으로 요약한 것입니다.

```
┌─────────────────────────────────────────────────────────────────────┐
│                    기존 RAG vs GraphRAG 비교                         │
├─────────────────────────────────────────────────────────────────────┤
│                                                                     │
│  [기존 RAG]                                                         │
│                                                                     │
│  문서 → 청킹 → 임베딩 → 벡터DB → 유사도 검색 → Top-K 청크 → LLM       │
│                                                                     │
│  문제점:                                                            │
│  - "이 회사의 전체 조직 구조는?" 같은 글로벌 질문에 약함               │
│  - 개별 청크만 보면 전체 그림을 못 봄                                 │
│  - 흩어진 정보를 종합하기 어려움                                      │
│                                                                     │
├─────────────────────────────────────────────────────────────────────┤
│                                                                     │
│  [GraphRAG with Community Summaries]                                │
│                                                                     │
│  문서 → 엔티티/관계 추출 → Knowledge Graph 구축                       │
│              ↓                                                      │
│        커뮤니티 탐지 (Leiden Algorithm)                               │
│              ↓                                                      │
│        각 커뮤니티를 LLM으로 요약 → Community Summaries               │
│              ↓                                                      │
│        계층적 요약 (Level 0, 1, 2, ...)                               │
│              ↓                                                      │
│        질문에 맞는 레벨의 요약 검색 → LLM                             │
│                                                                     │
│  장점:                                                              │
│  - 글로벌 질문에 강함 ("전체 요약해줘")                               │
│  - 계층적 이해 가능 (줌인/줌아웃)                                     │
│  - 연결된 정보를 함께 요약                                            │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

Part 1: 커뮤니티(Community)란?

그래프에서의 커뮤니티

```
┌─────────────────────────────────────────────────────────────────────┐
│                    Knowledge Graph의 커뮤니티                        │
│                                                                     │
```

```
│   커뮤니티 = 서로 밀접하게 연결된 노드들의 그룹                              │
│                                                                              │
│         ┌──────────── Community A ────────────┐                              │
│         │                                     │                              │
│         │    [김철수]──[이영희]               │                              │
│         │        │    \    │                  │                              │
│         │        │     \   │                  │                              │
│         │    [박민수]──[정수진]               │                              │
│         │             (개발팀)                │                              │
│         └─────────────────────────────────────┘                              │
│                          │                                                   │
│                     약한 연결                                                │
│                          │                                                   │
│         ┌──────────── Community B ────────────┐                              │
│         │                                     │                              │
│         │    [홍길동]──[김영수]               │                              │
│         │        │         │                  │                              │
│         │        │         │                  │                              │
│         │    [이민정]──[박서준]               │                              │
│         │             (마케팅팀)              │                              │
│         └─────────────────────────────────────┘                              │
│                                                                              │
│   - Community A: 개발팀 관련 엔티티들 (강하게 연결)                          │
│   - Community B: 마케팅팀 관련 엔티티들 (강하게 연결)                        │
│   - 두 커뮤니티는 약하게 연결됨                                              │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

Leiden Algorithm으로 커뮤니티 탐지

```python
# community_detection.py
import networkx as nx
import numpy as np

# 예시: 회사 조직 Knowledge Graph
G = nx.Graph()

# 개발팀 (Community A)
G.add_edges_from([
    ("김철수", "이영희"),
    ("김철수", "박민수"),
    ("이영희", "정수진"),
    ("박민수", "정수진"),
    ("김철수", "알파프로젝트"),
    ("이영희", "알파프로젝트"),
    ("알파프로젝트", "Python"),
    ("알파프로젝트", "FastAPI"),
])

# 마케팅팀 (Community B)
G.add_edges_from([
```

```
    ("홍길동", "김영수"),
    ("홍길동", "이민정"),
    ("김영수", "박서준"),
    ("이민정", "박서준"),
    ("홍길동", "브랜드캠페인"),
    ("김영수", "브랜드캠페인"),
])

# 팀 간 연결 (약한 연결)
G.add_edge("김철수", "홍길동")  # 협업

# Leiden 알고리즘으로 커뮤니티 탐지
# pip install leidenalg python-igraph
import igraph as ig
import leidenalg

# NetworkX → igraph 변환
g_igraph = ig.Graph.from_networkx(G)

# Leiden 알고리즘 실행
partition = leidenalg.find_partition(
    g_igraph,
    leidenalg.ModularityVertexPartition,
    seed=42
)

# 결과 출력
print("탐지된 커뮤니티:")
for i, community in enumerate(partition):
    members = [g_igraph.vs[idx]["_nx_name"] for idx in community]
    print(f"  Community {i}: {members}")
```

출력:

```
탐지된 커뮤니티:
  Community 0: ['김철수', '이영희', '박민수', '정수진', '알파프로젝트', 'Python',
'FastAPI']
  Community 1: ['홍길동', '김영수', '이민정', '박서준', '브랜드캠페인']
```

Part 2: Community Summary 생성

커뮤니티 → LLM 요약

```
┌──────────────────────────────────────────────────────────────────────┐
│                    Community Summary 생성 과정                        │
│                                                                       │
│  [1단계: 커뮤니티 내 정보 수집]                                        │
│                                                                       │
│  Community 0의 엔티티들:                                               │
```

```
│  - 김철수 (Tech Lead, 개발팀)                                          │
│  - 이영희 (Senior Developer)                                          │
│  - 박민수 (Junior Developer)                                          │
│  - 정수진 (DevOps Engineer)                                           │
│  - 알파프로젝트 (status: 진행중)                                        │
│  - Python, FastAPI                                                     │
│                                                                        │
│  Community 0의 관계들:                                                  │
│  - 김철수 MANAGES 알파프로젝트                                          │
│  - 김철수 COLLABORATES_WITH 이영희                                      │
│  - 알파프로젝트 USES Python                                             │
│  - ...                                                                 │
│                                                                        │
├────────────────────────────────────────────────────────────────────────┤
│                                                                        │
│  [2단계: LLM으로 요약 생성]                                             │
│                                                                        │
│  프롬프트:                                                              │
│  "다음 엔티티와 관계들을 분석하여 이 그룹의 핵심 내용을 요약하세요.     │
│   이 그룹이 무엇에 관한 것인지, 주요 엔티티와 그들의 관계,              │
│   핵심 활동이나 주제를 설명하세요."                                     │
│                                                                        │
│  [엔티티 목록]                                                          │
│  [관계 목록]                                                            │
│                                                                        │
├────────────────────────────────────────────────────────────────────────┤
│                                                                        │
│  [3단계: 생성된 Community Summary]                                      │
│                                                                        │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │ Community Summary #0: 개발팀 및 알파 프로젝트                     │  │
│  │                                                                  │  │
│  │ 이 커뮤니티는 ABC Corp의 개발팀을 중심으로 구성되어 있습니다.     │  │
│  │                                                                  │  │
│  │ 주요 구성원:                                                      │  │
│  │ - 김철수(Tech Lead): 알파 프로젝트를 관리하며 팀을 이끔           │  │
│  │ - 이영희(Senior Developer): 핵심 개발 담당                        │  │
│  │ - 박민수, 정수진: 개발 및 인프라 지원                             │  │
│  │                                                                  │  │
│  │ 핵심 활동:                                                        │  │
│  │ - 알파 프로젝트 개발 (현재 진행중)                                │  │
│  │ - Python과 FastAPI 기반 백엔드 개발                               │  │
│  │                                                                  │  │
│  │ 주요 특징:                                                        │  │
│  │ - 긴밀하게 협업하는 소규모 개발팀                                 │  │
│  │ - 현대적인 Python 기술 스택 사용                                  │  │
│  └──────────────────────────────────────────────────────────────────┘  │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

Community Summary 생성 코드

```python
# community_summary.py
from dataclasses import dataclass
from typing import List, Dict, Any
import anthropic

@dataclass
class CommunityInfo:
    """커뮤니티 정보"""
    id: int
    level: int  # 계층 레벨
    nodes: List[Dict[str, Any]]  # 노드 목록
    edges: List[Dict[str, Any]]  # 엣지 목록
    size: int  # 노드 수

@dataclass
class CommunitySummary:
    """커뮤니티 요약"""
    community_id: int
    level: int
    title: str
    summary: str
    key_entities: List[str]
    key_relationships: List[str]
    themes: List[str]
    token_count: int

class CommunitySummarizer:
    """커뮤니티 요약 생성기"""

    def __init__(self, llm_client):
        self.llm = llm_client

    def summarize(self, community: CommunityInfo) -> CommunitySummary:
        """커뮤니티를 요약"""

        # 1. 커뮤니티 정보를 텍스트로 변환
        context = self._build_context(community)

        # 2. LLM으로 요약 생성
        prompt = f"""다음은 Knowledge Graph에서 탐지된 커뮤니티(밀접하게 연결된 엔티티
그룹)입니다.
이 커뮤니티의 내용을 분석하고 요약해주세요.

{context}

다음 형식으로 응답해주세요:

제목: [이 커뮤니티를 설명하는 짧은 제목]

요약: [2-3문장으로 이 커뮤니티가 무엇에 관한 것인지 설명]

핵심 엔티티: [가장 중요한 엔티티 3-5개]
```

```
핵심 관계: [가장 중요한 관계 3-5개를 자연어로]

주요 테마: [이 커뮤니티의 주요 주제/테마 2-3개]"""

        response = self.llm.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=1000,
            messages=[{"role": "user", "content": prompt}]
        )

        result_text = response.content[0].text

        # 3. 응답 파싱
        return self._parse_response(result_text, community)

    def _build_context(self, community: CommunityInfo) -> str:
        """커뮤니티 정보를 컨텍스트 문자열로 변환"""
        lines = []

        # 노드 정보
        lines.append("## 엔티티 목록")
        for node in community.nodes:
            props = ", ".join(f"{k}: {v}" for k, v in node.get('properties',
{}).items())
            lines.append(f"- {node['name']} ({node['type']}): {props}")

        # 엣지 정보
        lines.append("\n## 관계 목록")
        for edge in community.edges:
            lines.append(f"- {edge['source']} --[{edge['type']}]-->
{edge['target']}")

        return "\n".join(lines)

    def _parse_response(self, text: str, community: CommunityInfo) ->
CommunitySummary:
        """LLM 응답 파싱"""
        # 간단한 파싱 (실제로는 더 정교하게)
        lines = text.split('\n')

        title = ""
        summary = ""
        key_entities = []
        key_relationships = []
        themes = []

        current_section = None
        for line in lines:
            line = line.strip()
            if line.startswith("제목:"):
                title = line.replace("제목:", "").strip()
            elif line.startswith("요약:"):
                summary = line.replace("요약:", "").strip()
                current_section = "summary"
```

```
            elif line.startswith("핵심 엔티티:"):
                key_entities = [e.strip() for e in line.replace("핵심 엔티티:",
"").split(",")]
            elif line.startswith("핵심 관계:"):
                current_section = "relationships"
            elif line.startswith("주요 테마:"):
                themes = [t.strip() for t in line.replace("주요 테마:",
"").split(",")]
            elif current_section == "relationships" and line.startswith("-"):
                key_relationships.append(line[1:].strip())

        return CommunitySummary(
            community_id=community.id,
            level=community.level,
            title=title,
            summary=summary,
            key_entities=key_entities,
            key_relationships=key_relationships,
            themes=themes,
            token_count=len(text) // 4,
        )


# 사용 예시
client = anthropic.Anthropic()
summarizer = CommunitySummarizer(client)

community = CommunityInfo(
    id=0,
    level=0,
    nodes=[
        {"name": "김철수", "type": "Person", "properties": {"role": "Tech Lead"}},
        {"name": "이영희", "type": "Person", "properties": {"role": "Senior
Dev"}},
        {"name": "알파프로젝트", "type": "Project", "properties": {"status": "진행
중"}},
        {"name": "Python", "type": "Technology", "properties": {}},
    ],
    edges=[
        {"source": "김철수", "target": "알파프로젝트", "type": "MANAGES"},
        {"source": "김철수", "target": "이영희", "type": "COLLABORATES_WITH"},
        {"source": "알파프로젝트", "target": "Python", "type": "USES"},
    ],
    size=4,
)

summary = summarizer.summarize(community)
print(f"제목: {summary.title}")
print(f"요약: {summary.summary}")
```

Part 3: 계층적 Community Summaries

다중 레벨 구조

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    계층적 Community Summaries                            │
│                                                                          │
│  Level 2 (최상위, 가장 추상적)                                            │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │ "ABC Corp은 기술 회사로, 개발팀과 마케팅팀이 협력하여          │    │
│  │  다양한 프로젝트를 수행하고 있다. 주로 Python 기반 기술과       │    │
│  │  디지털 마케팅에 집중하고 있다."                                 │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                              │                                           │
│                ┌─────────────┴──────────────┐                            │
│                ▼                            ▼                            │
│  Level 1 (중간)                                                          │
│  ┌────────────────────┐        ┌────────────────────┐                    │
│  │ "개발팀은 김철수가  │        │ "마케팅팀은 홍길동  │                  │
│  │  이끄는 팀으로,     │        │  팀장 하에 브랜드   │                  │
│  │  알파/베타 프로젝트 │        │  캠페인과 디지털    │                  │
│  │  를 진행 중이다."   │        │  마케팅을 담당한다." │                  │
│  └────────────────────┘        └────────────────────┘                    │
│          │                              │                                │
│     ┌────┴────┐                    ┌────┴────┐                          │
│     ▼         ▼                    ▼         ▼                          │
│  Level 0 (최하위, 가장 구체적)                                            │
│  ┌────────┐ ┌────────┐        ┌────────┐ ┌────────┐                     │
│  │알파팀  │ │베타팀  │        │캠페인팀 │ │SNS팀   │                    │
│  │상세정보 │ │상세정보 │        │상세정보 │ │상세정보 │                    │
│  └────────┘ └────────┘        └────────┘ └────────┘                     │
│                                                                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [질문 유형별 레벨 선택]                                                  │
│                                                                          │
│  "회사 전체 개요는?" → Level 2 사용 (가장 추상적)                         │
│  "개발팀이 뭐해?" → Level 1 사용 (중간)                                   │
│  "김철수가 쓰는 기술은?" → Level 0 사용 (가장 구체적)                     │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

계층적 요약 생성

```python
# hierarchical_communities.py
from typing import List, Dict
from dataclasses import dataclass

@dataclass
class HierarchicalSummary:
    """계층적 커뮤니티 요약"""
    level: int
```

```python
    summaries: List[CommunitySummary]

class HierarchicalCommunitySummarizer:
    """계층적 커뮤니티 요약 생성"""

    def __init__(self, llm_client, max_levels: int = 3):
        self.llm = llm_client
        self.max_levels = max_levels
        self.base_summarizer = CommunitySummarizer(llm_client)

    def build_hierarchy(self,
                        graph,
                        resolution_levels: List[float] = [1.0, 0.5, 0.2]
                        ) -> List[HierarchicalSummary]:
        """
        계층적 커뮤니티 요약 생성

        resolution_levels: 높을수록 작은 커뮤니티, 낮을수록 큰 커뮤니티
        """
        all_levels = []

        for level, resolution in enumerate(resolution_levels):
            # 해당 해상도로 커뮤니티 탐지
            communities = self._detect_communities(graph, resolution)

            # 각 커뮤니티 요약
            summaries = []
            for comm in communities:
                if level == 0:
                    # Level 0: 개별 커뮤니티 직접 요약
                    summary = self.base_summarizer.summarize(comm)
                else:
                    # Level 1+: 하위 레벨 요약들을 종합
                    summary = self._summarize_from_children(
                        comm, all_levels[level - 1].summaries
                    )
                summaries.append(summary)

            all_levels.append(HierarchicalSummary(
                level=level,
                summaries=summaries
            ))

        return all_levels

    def _detect_communities(self, graph, resolution: float) ->
List[CommunityInfo]:
        """특정 해상도로 커뮤니티 탐지"""
        import igraph as ig
        import leidenalg

        g_igraph = ig.Graph.from_networkx(graph)

        partition = leidenalg.find_partition(
```

```
            g_igraph,
            leidenalg.RBConfigurationVertexPartition,
            resolution_parameter=resolution,
        )

        communities = []
        for i, members in enumerate(partition):
            nodes = []
            for idx in members:
                node_id = g_igraph.vs[idx]["_nx_name"]
                nodes.append({
                    "name": node_id,
                    "type": graph.nodes[node_id].get("type", "Unknown"),
                    "properties": graph.nodes[node_id],
                })

            # 커뮤니티 내부 엣지
            edges = []
            member_set = set(g_igraph.vs[idx]["_nx_name"] for idx in members)
            for u, v in graph.edges():
                if u in member_set and v in member_set:
                    edges.append({
                        "source": u,
                        "target": v,
                        "type": graph.edges[u, v].get("type", "RELATED"),
                    })

            communities.append(CommunityInfo(
                id=i,
                level=0,
                nodes=nodes,
                edges=edges,
                size=len(nodes),
            ))

        return communities

    def _summarize_from_children(self,
                                 community: CommunityInfo,
                                 child_summaries: List[CommunitySummary]
                                 ) -> CommunitySummary:
        """하위 레벨 요약들을 종합하여 상위 레벨 요약 생성"""

        # 이 커뮤니티에 속하는 하위 요약 찾기
        community_node_names = {n["name"] for n in community.nodes}
        relevant_summaries = []

        for child in child_summaries:
            # 하위 요약의 엔티티가 현재 커뮤니티에 포함되는지
            if any(e in community_node_names for e in child.key_entities):
                relevant_summaries.append(child)

        if not relevant_summaries:
            return self.base_summarizer.summarize(community)
```

```
        # 하위 요약들을 종합
        prompt = f"""다음은 여러 하위 그룹의 요약입니다.
이들을 종합하여 하나의 통합된 상위 레벨 요약을 만들어주세요.

하위 그룹 요약들:
"""
        for i, summary in enumerate(relevant_summaries):
            prompt += f"""
그룹 {i+1}: {summary.title}
{summary.summary}
핵심 엔티티: {', '.join(summary.key_entities)}
"""

        prompt += """
위 그룹들을 종합한 상위 레벨 요약을 작성해주세요.
더 추상적이고 전체적인 관점에서 요약해주세요.

제목: [통합 제목]
요약: [통합 요약 2-3문장]
핵심 엔티티: [가장 중요한 엔티티들]
주요 테마: [전체를 관통하는 테마]"""

        response = self.llm.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=800,
            messages=[{"role": "user", "content": prompt}]
        )

        # 파싱 후 반환 (간소화)
        return CommunitySummary(
            community_id=community.id,
            level=community.level,
            title="통합 요약",
            summary=response.content[0].text,
            key_entities=[],
            key_relationships=[],
            themes=[],
            token_count=len(response.content[0].text) // 4,
        )
```

Part 4: Community Summaries를 RAG에 활용

질문 유형별 검색 전략

```
┌──────────────────────────────────────────────────────────────────────┐
│                  Community Summaries 기반 RAG                        │
│                                                                      │
│  [Local Questions - 구체적 질문]                                     │
│  "김철수가 어떤 기술을 사용하나요?"                                  │
│                                                                      │
```

```
│  → 기존 방식: 엔티티 기반 서브그래프 추출                                  │
│  → 또는: Level 0 (가장 상세한) Community Summary 검색                    │
│                                                                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [Global Questions - 전체적 질문]                                        │
│  "이 회사의 주요 사업 분야는 무엇인가요?"                                  │
│                                                                          │
│  → Level 2 (최상위) Community Summaries 모두 사용                        │
│  → Map-Reduce 방식으로 종합                                              │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │                    Map-Reduce 과정                               │   │
│  │                                                                  │   │
│  │  [Summary A] [Summary B] [Summary C] ...  (Level 2 전체)         │   │
│  │       │           │           │                                  │   │
│  │       ▼           ▼           ▼                                  │   │
│  │  [Partial    [Partial    [Partial      (각각 LLM에 질문)         │   │
│  │   Answer A]   Answer B]   Answer C]                              │   │
│  │       │           │           │                                  │   │
│  │       └───────────┴───────────┘                                  │   │
│  │                   │                                              │   │
│  │                   ▼                                              │   │
│  │           [Final Answer]  (종합)                                 │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

GraphRAG 검색 구현

```python
# graphrag_retriever.py
from typing import List, Tuple
from enum import Enum

class QueryType(Enum):
    LOCAL = "local"    # 구체적 질문
    GLOBAL = "global"  # 전체적 질문

class GraphRAGRetriever:
    """GraphRAG 검색기"""

    def __init__(self,
                 community_summaries: List[HierarchicalSummary],
                 subgraph_extractor,
                 llm_client):
        self.summaries = community_summaries
        self.extractor = subgraph_extractor
        self.llm = llm_client

    def retrieve(self, query: str) -> Tuple[str, QueryType]:
        """질문에 맞는 컨텍스트 검색"""
```

```
        # 1. 질문 유형 분류
        query_type = self._classify_query(query)

        if query_type == QueryType.LOCAL:
            # 로컬 질문: 엔티티 기반 또는 Level 0 검색
            context = self._local_search(query)
        else:
            # 글로벌 질문: 상위 레벨 요약들 사용
            context = self._global_search(query)

        return context, query_type

    def _classify_query(self, query: str) -> QueryType:
        """질문 유형 분류"""
        global_keywords = [
            "전체", "모든", "요약", "개요", "주요",
            "어떤 것들", "무엇들", "종류", "분야",
            "overall", "summary", "main", "all"
        ]

        query_lower = query.lower()
        for keyword in global_keywords:
            if keyword in query_lower:
                return QueryType.GLOBAL

        return QueryType.LOCAL

    def _local_search(self, query: str) -> str:
        """로컬 검색: 구체적 정보 검색"""
        # Level 0 요약 중 관련된 것 찾기
        level_0 = self.summaries[0]  # 가장 상세한 레벨

        # 쿼리와 관련된 커뮤니티 찾기 (간단한 키워드 매칭)
        relevant_summaries = []
        query_words = set(query.lower().split())

        for summary in level_0.summaries:
            summary_words = set(summary.summary.lower().split())
            summary_words.update(e.lower() for e in summary.key_entities)

            overlap = query_words & summary_words
            if overlap:
                relevant_summaries.append((len(overlap), summary))

        # 관련도 순 정렬
        relevant_summaries.sort(key=lambda x: x[0], reverse=True)

        # 상위 3개 사용
        context_parts = []
        for _, summary in relevant_summaries[:3]:
            context_parts.append(f"""
### {summary.title}
{summary.summary}
```

```
핵심 엔티티: {', '.join(summary.key_entities)}
핵심 관계: {', '.join(summary.key_relationships[:3])}
""")

        return "\n".join(context_parts)

    def _global_search(self, query: str) -> str:
        """글로벌 검색: 전체 요약 기반 Map-Reduce"""
        # 최상위 레벨 사용
        top_level = self.summaries[-1]

        # Map: 각 요약에 대해 부분 답변 생성
        partial_answers = []

        for summary in top_level.summaries:
            prompt = f"""다음 정보를 바탕으로 질문에 관련된 내용만 추출해주세요.

정보:
{summary.summary}
핵심 엔티티: {', '.join(summary.key_entities)}
주요 테마: {', '.join(summary.themes)}

질문: {query}

관련된 내용만 간결하게 답변하세요. 관련 없으면 "관련 정보 없음"이라고 하세요."""

            response = self.llm.messages.create(
                model="claude-sonnet-4-20250514",
                max_tokens=300,
                messages=[{"role": "user", "content": prompt}]
            )

            answer = response.content[0].text
            if "관련 정보 없음" not in answer:
                partial_answers.append(answer)

        # Reduce: 부분 답변들 종합
        if not partial_answers:
            return "관련 정보를 찾을 수 없습니다."

        combined = "\n\n".join(f"- {ans}" for ans in partial_answers)

        return f"""[글로벌 검색 결과 - {len(partial_answers)}개 커뮤니티에서 정보 수
집]

{combined}"""

    def answer(self, query: str) -> str:
        """질문에 답변"""
        context, query_type = self.retrieve(query)

        if query_type == QueryType.GLOBAL:
            # 글로벌: 이미 Map-Reduce 결과가 컨텍스트
```

```
            prompt = f"""다음 정보를 종합하여 질문에 답변해주세요.

{context}

질문: {query}

종합적이고 완전한 답변을 제공하세요."""
        else:
            # 로컬: 컨텍스트 기반 답변
            prompt = f"""다음 정보를 바탕으로 질문에 답변해주세요.

{context}

질문: {query}

구체적이고 정확한 답변을 제공하세요."""

        response = self.llm.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=1000,
            messages=[{"role": "user", "content": prompt}]
        )

        return response.content[0].text
```

Part 5: 정리

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    Community Summaries 정리                              │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [What] Community Summaries란?                                           │
│  - Knowledge Graph에서 밀접하게 연결된 노드 그룹(커뮤니티)의 요약        │
│  - LLM이 각 커뮤니티의 엔티티/관계를 분석하여 생성                       │
│  - 계층적 구조로 여러 레벨 (상세 ↔ 추상)                                 │
│                                                                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [Why] 왜 필요한가?                                                      │
│  - 기존 RAG: 개별 청크만 검색 → 전체 그림 못 봄                          │
│  - GraphRAG: 연결된 정보를 그룹으로 요약 → 글로벌 질문 가능              │
│                                                                          │
│  예: "이 회사의 주요 사업 분야는?"                                       │
│      기존 RAG: 여러 청크 조각 반환 (불완전)                              │
│      GraphRAG: 상위 레벨 요약으로 전체 답변 가능                         │
│                                                                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [How] 어떻게 만드나?                                                    │
│                                                                          │
│  1. Knowledge Graph 구축 (엔티티 + 관계)                                 │
```

```
│  2. Leiden Algorithm으로 커뮤니티 탐지                                    │
│  3. 각 커뮤니티를 LLM으로 요약                                            │
│  4. 해상도 조절하여 계층적 요약 생성                                      │
│                                                                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [When] 언제 사용하나?                                                    │
│                                                                          │
│  - Local Query: 서브그래프 또는 Level 0 요약                              │
│  - Global Query: 상위 레벨 요약 + Map-Reduce                              │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────────────┐
│                        기존 RAG vs GraphRAG 비교                         │
├──────────────┬──────────────────────┬──────────────────────────────────┤
│              │      기존 Vector RAG  │          GraphRAG                │
├──────────────┼──────────────────────┼──────────────────────────────────┤
│ 인덱싱       │ 청크 → 임베딩        │ 그래프 + 커뮤니티 요약           │
├──────────────┼──────────────────────┼──────────────────────────────────┤
│ 검색 단위    │ 개별 청크            │ 서브그래프 / 커뮤니티 요약       │
├──────────────┼──────────────────────┼──────────────────────────────────┤
│ 로컬 질문    │ ☑ 좋음               │ ☑ 좋음                           │
├──────────────┼──────────────────────┼──────────────────────────────────┤
│ 글로벌 질문  │ ✕ 약함               │ ☑ 강함                           │
├──────────────┼──────────────────────┼──────────────────────────────────┤
│ 구축 비용    │ 낮음                 │ 높음 (LLM 호출 많음)             │
├──────────────┼──────────────────────┼──────────────────────────────────┤
│ 적합 케이스  │ 단순 Q&A             │ 분석, 요약, 전체 이해 필요       │
└──────────────┴──────────────────────┴──────────────────────────────────┘
```